Memento SQL

Création de tables.

```
CREATE TABLE Commentaire(
    idcomm NUMBER(3) CONSTRAINT pk_comm PRIMARY KEY,
    email VARCHAR2(50) CONSTRAINT fk_internaute REFERENCES Internaute(email),
    idparent NUMBER(3) CONSTRAINT fk_comm REFERENCES Commentaire(idcomm),
    idfilm NUMBER(3), --NUMBER(n,p): n chiffres au total, p apres la virgule
    datecomm DATE CONSTRAINT c_datecomm NOT NULL,
    CONSTRAINT fk_notation FOREIGN KEY (email, idfilm) REFERENCES Notation(email, idfilm),
    CONSTRAINT c_idfilm CHECK (idfilm >= 0)
);
-- CREATE combine a un SELECT: copie les donnees mais pas les contraintes
CREATE TABLE test AS SELECT * FROM dual;
ALTER TABLE test ADD(
  CONSTRAINT pk_dummy PRIMARY KEY(dummy),
  CONSTRAINT c_idfilm_test CHECK (idfilm >= 1)
);
ALTER TABLE test MODIFY idfilm NUMBER(4); --changer le type
ALTER TABLE test DROP COLUMN coucou;
DROP TABLE test;
```

Insertion, suppression, mise à jour.

```
INSERT INTO film VALUES (2, 'toto', to_date('20/12/2010'), 'policier', 110, 'Monaco', null);
INSERT INTO film(idfilm, titre) VALUES (3, 'toto 2');
INSERT INTO film(idfilm, titre, duree) SELECT idfilm+3, titre,duree+10 FROM film;

UPDATE film SET entree = 50000 WHERE idfilm = 1;
UPDATE film f1 SET entree = (select entree+duree from film f2 where f1.idfilm = f2.idfilm);

DELETE FROM film where idfilm = 1;
DELETE FROM film where idfilm in (select idfilm from film where genre is null);
```

Interrogation simple, tri, date.

```
SELECT titre, paysfilm, genre FROM film WHERE paysfilm = 'France' OR genre = 'policier';

SELECT titre FROM film WHERE titre LIKE 'Le%'; --nombre de caractere quelconcque
SELECT titre FROM film WHERE titre LIKE 'Av_tar'; --exactement 1 caractere

SELECT * FROM film ORDER BY entree; --croissant, pour decroissant, ajouter DESC
SELECT * FROM film ORDER BY paysfilm, entree;
SELECT round(entree, -2) arrondi FROM film ORDER BY arrondi; --alias

SELECT To_Char (datesortie, 'DD/MM/YYYY') FROM film;
SELECT To_Char (datesortie, 'DD month YYYY') FROM film;
SELECT titre FROM film WHERE To_Char (datesortie, 'MM') = '10'; --films sortis en octobre
SELECT titre FROM film WHERE To_Date ('16-12-09', 'DD-MM-YY') = datesortie;
```

Jointure.

```
SELECT jouer.idfilm FROM film INNER JOIN jouer ON film.idfilm=jouer.idfilm WHERE idacteur=2;
SELECT idfilm FROM film NATURAL JOIN jouer WHERE idacteur = 2;
SELECT f1.entree-f2.entree FROM film f1,film f2 WHERE f1.titre='Toto' AND f2.titre='Avatar';
SELECT nom FROM intrnte LEFT JOIN nttion ON intrnte.email=nttion.email WHERE note IS NULL;
```

Structure arborescente.

```
--parcours de la racine aux feuilles
SELECT idcomm FROM commentaire START WITH idparent IS NULL CONNECT BY idparent=PRIOR idcomm;
--parcours d'une branche depuis la feuille/noeud interne
SELECT idcomm FROM commentaire START WITH idcomm=4 CONNECT BY PRIOR idparent = idcomm;
--level: niveau dans l'arbre
SELECT idcomm, RPAD(' ', level-1) || texte FROM commentaire START WITH idparent IS NULL and
    email ='Juliette.Luna@gmail.fr' CONNECT BY idparent = PRIOR idcomm;
```

Operations ensemblistes.

```
--sur des relations de meme schema
SELECT titre FROM film WHERE paysfilm = 'France'
UNION --INTERSECT, MINUS
SELECT titre FROM film WHERE paysfilm = 'USA';
```

Fonctions d'agrégations, clause HAVING.

```
SELECT max(entree) FROM film; --min, avg, sum
SELECT count(*) FROM film; --compte toutes les lignes
SELECT count(entree) FROM film; --compte les lignes pour lesquelles entree est not null
SELECT count(DISTINCT genre) FROM film;

SELECT count(*) FROM film WHERE paysfilm = 'France';
SELECT max(entree) FROM film WHERE paysfilm = 'France';

--les attributs dans le SELECT hors fonctions count, max ...doivent etre dans le GROUP BY
SELECT paysfilm, genre, count(*) FROM film GROUP BY paysfilm, genre;
-- on peut mettre plus d'arguments dans le GROUP BY que dans le SELECT
SELECT nom, count(*) FROM  acteur NATURAL JOIN jouer GROUP BY nom,idacteur;

SELECT paysfilm, sum(entree*10) CA FROM film GROUP BY paysfilm ORDER BY CA;
SELECT max(avg(note)) FROM notation GROUP BY idfilm;

SELECT paysfilm, COUNT(*) FROM film GROUP BY paysfilm HAVING COUNT(*)>2;
```

Sous requête.

```
-- Films francais sortis la meme annee que "Toto"
SELECT titre FROM film WHERE paysfilm = 'France' AND TO_CHAR(datesortie, 'YYYY') =
(SELECT TO_CHAR(datesortie, 'YYYY') FROM film WHERE titre = 'Toto');
-- Acteur le plus vieux
SELECT nom FROM acteur WHERE datenaiss = (SELECT min(datenaiss) FROM acteur);
-- Email des internautes ayant note les memes films qu'Amandine Dahan (NOT IN possible)
SELECT titre, email FROM film NATURAL JOIN notation
WHERE idfilm IN (SELECT idfilm FROM notation WHERE email='Amandine.Dahan@gmail.fr');
--internaute qui a note tous les films
SELECT email FROM notation GROUP BY email
HAVING count(*) = (SELECT count(*) FROM film);

-- Titre des films qui ont ete notes par un internaute de la meme nationalite que le film
SELECT titre FROM film WHERE EXISTS
(SELECT * FROM internaute NATURAL JOIN notation WHERE paysfilm = paysint
 AND idfilm = notation.idfilm);

--noms des acteurs qui ont joue avec un acteur plus jeune
SELECT nom FROM acteur a1 INNER JOIN jouer j1 ON a1.idacteur=j1.idacteur WHERE EXISTS
(SELECT * FROM acteur a2 INNER JOIN jouer j2 ON a2.idacteur = j2.idacteur
 WHERE j1.idfilm = j2.idfilm AND a1.datenaiss < a2.datenaiss);
```

Ordre des mots-clés.

```
SELECT ...
FROM ... [INNER JOIN ... ON ...]
WHERE ...
[GROUP BY ...]
[HAVING ...]
[ORDER BY ...];
```